

B144 Software Design and Modeling

Individual Project Report

Academy— Gamified Programming Learning Platform

Student 1: Ravshanjon Rakhmonberdiev | ID: GH1047114

Student 2: Aihem Geswila | ID: GH1043158

GitHub Repository:

https://github.com/Ravshanjohn/group_project/tree/main/software%20design%20and%20modellin%20g

Video presentation:

https://drive.google.com/file/d/1e_124-ic18mkIXYGGLfrEqC83Ok00rMx/view?usp=drive_link

1. Team Introduction

Our team consists of two members. Ravshan takes the backend developer and project planner parts. This means Ravshan is responsible for designing the database, server-side logic, setting up the project timeline, and handling tasks on GitHub Projects. Aihem Geswila takes the role of frontend developer and documentation lead. Meaning Aihem is responsible for building the user interface(UI), integrating api(s), and writing the daily project report.

Both members share some tasks equally. These tasks include brainstorming the project idea, reviewing each other's code through pull requests and meeting if needed, and testing the final delivered code. We chose to split work that way because it matches each person's own strengths while making nobody works alone on any major part.

We communicate through [WhatsApp] for daily messages and hold weekly video calls on [Google meet] to look through progress. We use GitHub Projects as our main task to track what needs to be done, what is finished.

2. Idea Pitching

2.1 The Problem

Many people who want to learn programming language feel lost what to choose and practice. There are too many resources online out there, and most of them are not related with each other. A student having watched a video on one website, having exercised on another, and played coding games on a third. There is no single place that combines all of these in one structural way. On top of that, most good learning paths are not free, and free options are incomplete or have no clear direction usually.

2.2 Our Solution

Our web application is called Academy. It is a programming language learning platform that has many languages with its own structured roadmap. The roadmap provide learners with structured steps, from beginner topics to advanced ones, through exercises, videos, and interactive contents.

The platform has two versions. The free version gives for users opportunity to a select set of exercises taken from the paid roadmaps. This lets learners try out the very platform and see if it works for them before making a payment. The paid version unlocks the full roadmap with all available exercises, videos, and learning materials for a chosen programming language.

2.3 The Gamification System

What makes our platform unique is the gaming part. Users can play small specific games (related to coding) and earn experience points, which we call XP. These games are enjoyable on their own, but they also serve a bigger purpose. The games are some of examples of what students can learn during the our paid roadmaps. When users collect enough XP while playing, they can use it to unlock specific exercises from the paid roadmaps for completely free. This gives students a reason to keep coming back and lust for practising.

2.4 Personalized Recommendations

The platform also tracks what performance of each user. Based on their past activities, the system gives personalized feedback for the past user activity. It tells to users what topics they are struggling at, what they are good at, and what they should practise more. By doing it, every learner gets path that fits their own level and speed.

2.5 Target Users

Our main users are from beginner to intermediate programmers, especially students who want to learn a new language in a affordable and structured way.

3. Project Planning

3.1 Timeline

We split our project into four main parts across the module weeks:

Phase	Weeks	Activities
Planning	Weeks 1–2	Brainstorming, team charter creation, setting up GitHub repository, writing initial user stories
Design	Weeks 3–5	Creating wireframes, defining requirements, drawing UML diagrams, designing the database
Development	Weeks 5–8	Building the frontend and backend, connecting the database, integrating the reusable library
Testing & Submission	Weeks 9–10	Testing features, fixing bugs, recording demo video, writing the report

3.2 Task Distribution

We used GitHub Projects with a Kanban board to manage our tasks. Each task was created as a GitHub Issue on a Linear and assigned to one team member. Tasks moved through three columns: To Do, In Progress, and Done. This made easy for both of us to see what we were working on.

3.3 Tools Used

- GitHub — for version control and hosting the code repository
- GitHub Projects — for task management using a Kanban board
- [Discord/WhatsApp] — for daily communication
- [Figma/draw.io] — for designing wireframes and UML diagrams
- [VS Code] — as our code editor
- Linear – for issue assignment
- Google meet – video representation and communication

3.4 Git Strategy

We followed a branch-based workflow. We keep them main branch always as a working code. For every new feature, we created a separate branch named after that feature, for example "feature/xp-system" or "feature/recommendation-engine". Having made the very feature, we opened a pull request so the other person could review it before merging them.

4. Requirements Elicitation

4.1 Stakeholders

We set the following stakeholders for our platform:

- Students (primary users) — they use the platform to learn programming through exercises, games, and roadmaps
- Content creators / administrators — they manage the exercises, videos, and roadmaps on the web app
- The development team (us) — we build and maintain the application

4.2 User Stories

We wrote stories of users to capture what each type of user needs. User stories follow the format: "As a [role], I want to [action], so that [benefit]."

Student stories:

- As a student, I want to try out free exercises, so that I can try the platform before paying.
- As a student, I want to follow one specific roadmap for a particular language, so that I learn topics in the right order.
- As a student, I want to play coding games and earn XP, so that I can unlock exclusive exercises for free.
- As a student, I want to see recommendations based on my past progresses, so that I know what to study next or practice more.
- As a student, I want to see my progress across exercises, so that I can look up how far I have come.

Administrator stories:

- As an admin, I want to add new exercises to a roadmap, so that the content stays up to date.
- As an admin, I want to change exercise statys from free to paid or vice versa, so that I can control what users see without paying.
- As an admin, I want to see how many users completed each exercise, so that I can find that given exercises that are too hard or too easy.

4.3 Use Cases

We created use case descriptions for the main interactions:

Use Case 1: Complete an Exercise

Field	Description
ID	UC-01
Name	Complete an exercise
Actor	Student
Pre-condition	Student is logged in and has access to the exercise (free or unlocked)
Event flow	1. Student opens a roadmap. 2. Student selects an exercise. 3. System shows the exercise content. 4. Student writes and submits an answer. 5. System checks the answer and shows result.
Post-condition	Exercise is marked as completed in the student's profile
Alternative case	If the answer is wrong, the system shows a hint and lets the student try again

Use Case 2: Play a Game and Earn XP

Field	Description
ID	UC-02
Name	Play a game
Actor	Student
Pre-condition	Student is logged in
Event flow	1. Student opens the games section. 2. Student picks a game and difficulty level. 3. Student plays the game. 4. System calculates the score. 5. System awards XP based on the score.
Post-condition	XP is added to the student's total, attempt is recorded
Alternate case	If the student already played at this difficulty, the old score is updated only if the new one is higher from the current

4.4 MoSCoW Prioritization

Priority	Features
Must Have	User registration and login, free exercises, exercise completion tracking, XP system
Should Have	Paid roadmaps, game scoring with difficulty levels, progress dashboard
Could Have	Personalized recommendations, video content within exercises
Won't Have (for now)	Social features like leaderboards, desktop and mobile version, payment gateway

5. Refinement and Specification

5.1 Refined Functional Requirements

After writing our user stories and use cases, we refined them into functional requirements. Each requirement is specific, testable, and tied to feature.

ID	Requirement	Related User Story
FR-01	The system shall allow users to register with an email and password	Student registration
FR-02	The system shall display a list of available programming languages, each with its own roadmap	Browse roadmaps
FR-03	The system shall let users open and complete exercises that are marked as free	Access free exercises
FR-04	The system shall check the user's submitted answer and return correct or incorrect	Complete an exercise
FR-05	The system shall award XP to users based on their game score and difficulty level	Play a game and earn XP
FR-06	The system shall allow users to unlock paid exercises by spending their earned XP	Unlock with XP
FR-07	The system shall track which exercises each user has started, completed, or not attempted	Track progress
FR-08	The system shall generate recommendations based on the user's completed and failed exercises	Get recommendations

5.2 Non-Functional Requirements

ID	Requirement	Category
NFR-01	The application shall respond to user actions within 2 seconds	Performance
NFR-02	User passwords shall be stored using hashing, not plain text	Security
NFR-03	The user interface shall be clear and usable without a tutorial	Usability
NFR-04	The system shall handle at least 50 users at the same time without slowing down	Scalability
NFR-05	The codebase shall follow SOLID principles to allow easy future changes	Maintainability

5.3 Acceptance Criteria Example

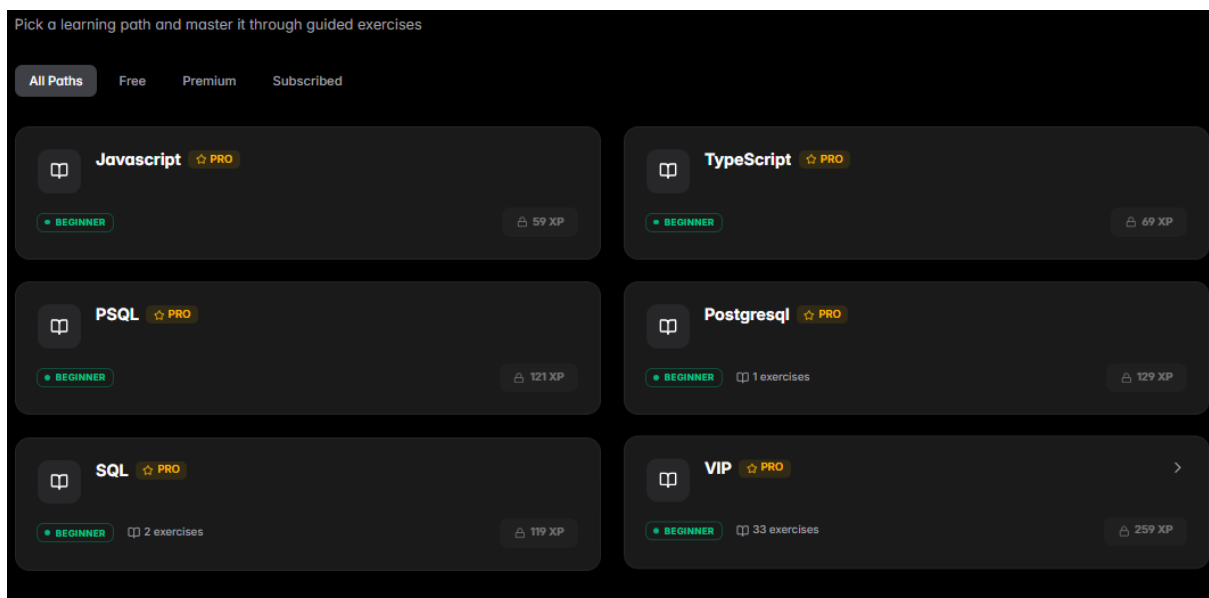
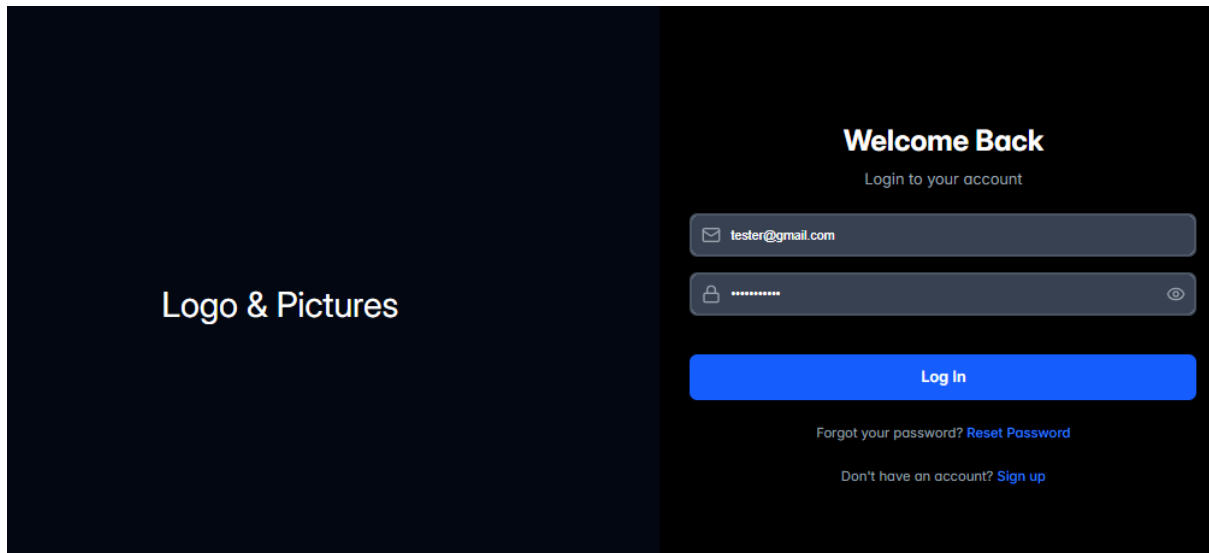
To make sure each requirement is truly done, we wrote acceptance criteria. Here is one example:

For FR-05 (Award XP from games):

- The system awards XP only after a game session is finished.

- XP amount depends on the score and the chosen difficulty level.
- If the user replays a game at the same difficulty, XP is only updated if the new score is higher than the old one.
- The updated XP total is shown on the user's profile right after the game finished.

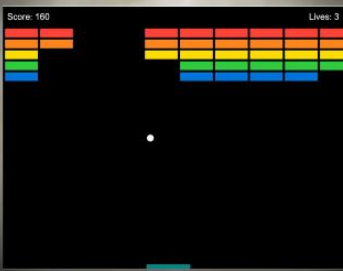
5.4 Wireframes



Earn GP by playing games

Select a game to start playing

2560 XP



Score: 100 Lives: 3

Breakout

Easy Medium Hard

Select



Snake

SCORE: 9 RESTART

Snake

Easy Medium Hard

Select



Tic-Tac-Toe

Easy Medium Hard

Select

Scores About Difficulty

Leaderboard

1 Tester a

Easy

Medium

Hard

Tic-Tac-Toe

Choose your opponent



vs AI

Easy difficulty

Coming Soon



vs Human

Play against the AI and challenge yourself across different difficulty levels. Track your wins and compete for the highest score!

Account

Profile

Certificates

Progress

Balance

Profile

Logout

Avatar

Name

Tester

Last Name

a

Email

tester@gmail.com

Balance

2560 XP

6. Modelling and Architecture

6.1 High-Level Architecture

The application has a three main architecture, which the system separated into three different layers: the presentation layer, the business logic layer, and the data layer. This separation is a approach in software engineering field because it makes the system easier to maintain, test, and scale (Sommerville, 2016). Each layer has a own clear responsibility and communicates only with the layer directly below or above it.

Layer	Responsibility	Technologies
Presentation Layer	Handles the user interface and displays data to the user. It includes all screens such as login page, roadmap view, exercise screen, game screen, and progress dashboard.	[JavaFX / Electron / Python Tkinter]
Business Logic Layer	Contains the main application logic. These are the XP calculation engine, the recommendation algorithm, exercise validation, game scoring, and user authentication.	[Python / Java / C#]
Data Layer	Manages data storage and retrieval. Handles all database process of operation including storing user	[SQLite / PostgreSQL]

	profiles, exercise information, game scores, progress tracking, and roadmap content.	
--	--	--

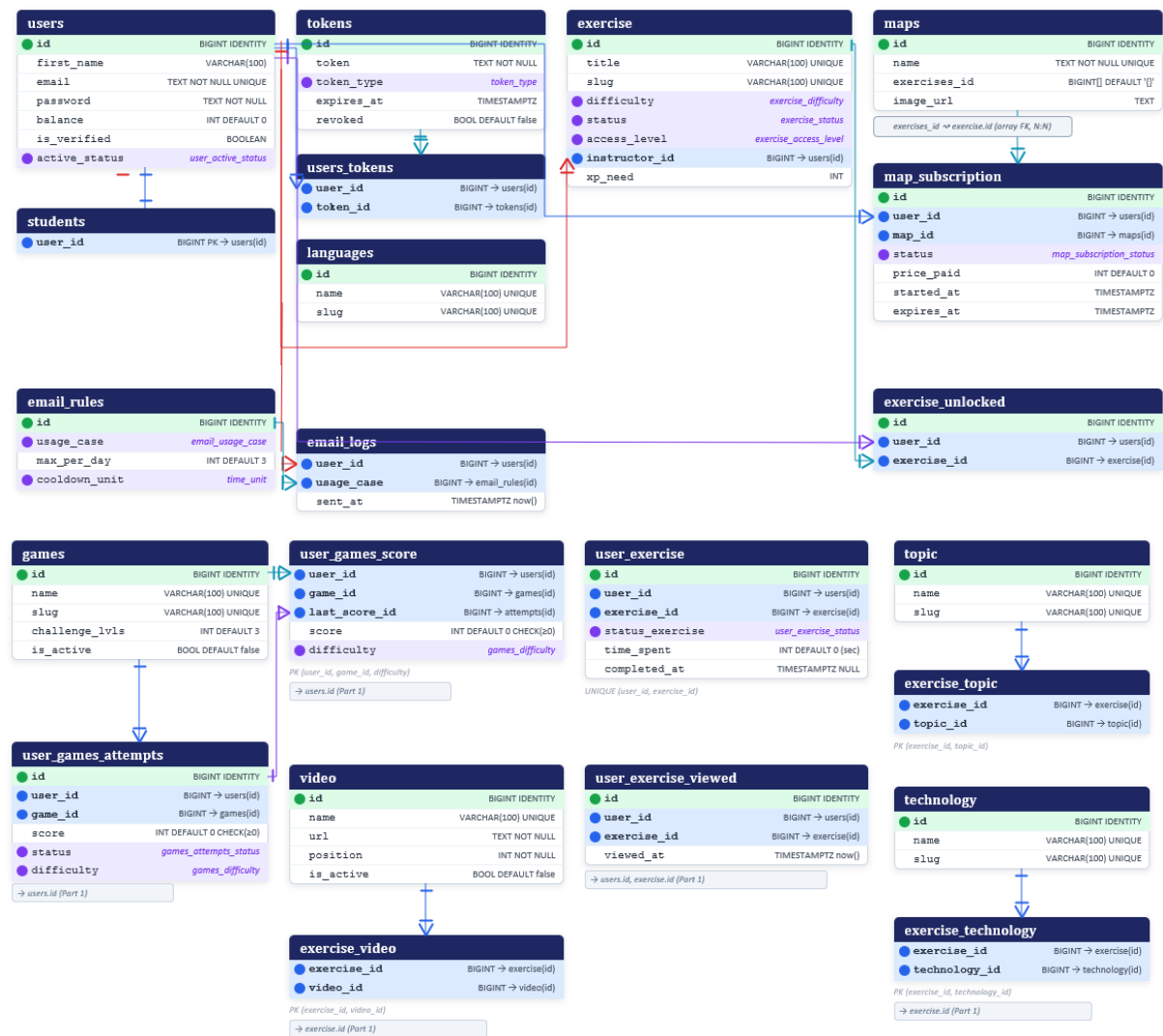
This architecture makes easy us to modify the user interface later without having an effect the business logic and database. Also, if we want to switch from SQLite to PostgreSQL, we only change the data layer. This flexibility makes our system easier to maintain and scale in the long term.

6.2 Low-Level Architecture

At a lower level, the application is divided into modules that based on its main features. Each module contains a group of related classes that handle specific area of functionality.

Module	Key Classes	Purpose
Auth Module	User, AuthService, PasswordHasher	Handles registration, login, and management of session
Roadmap Module	Roadmap, Topic, Exercise, ExerciseService	Manages language roadmaps, topics, and exercises
Game Module	Game, GameSession, ScoreCalculator	Handles game logic, scoring, and levels of difficulty
XP Module	XPManager, UnlockService	Calculates XP rewards and manages content unlocking
Recommendation Module	RecommendationEngine, UserProfile, ActivityTracker	Analyzes user activity and generates personalized suggestions
Admin Module	AdminPanel, ContentManager	Allows administrators to manage exercises and roadmaps

6.3 Class Diagram

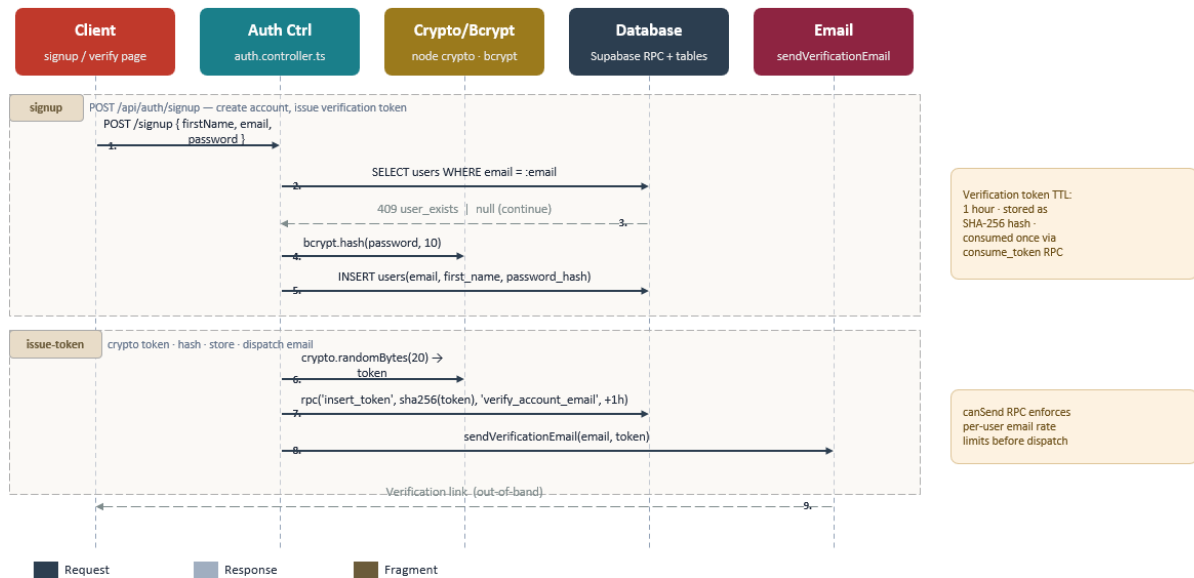


The class diagram shows the key classes in our system and how they relate with each other. The User class is at the beginning of class diagram. Each User has a list of completed exercises (tracked through the user_exercise class), a total XP score, and an exercise type (free or paid depending on xp_need). The Roadmap class contains an group of list Topics, and each Topic contains multiple Exercise objects. The Exercise class has attributes for difficulty level, programming language, content, and whether it is free or paid.

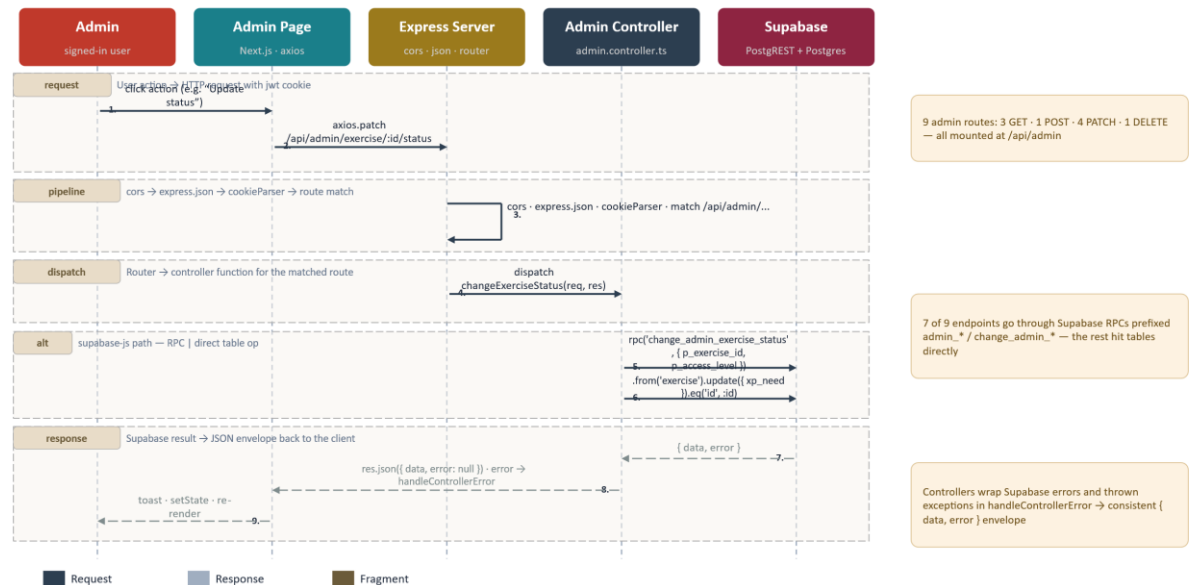
The Game class represents each coding game. It has a difficulty level and a category. The user_games_score class links a User to a Game and records the score, XP earned. The RecommendationEngine class takes a user_exercise and user_exercise_viewed (which summarises a user's strengths and weaknesses) and returns a list of suggested exercises or topics

6.4 Sequence Diagram

Signup & Email Verification



Admin Page — Request Processing in the Backend



Sequence Diagram 1: Signup and Email Verification

The former sequence diagram shows what happens when a new user creates an account on our platform. The flow starts from the Client, which is the signup page. When the user fills the forms in their first name, email, and password and clicks the signup button, the Client sends a POST request to the Auth Controller. The Auth Controller first queries the Database to check if the email already exists or not. If it does exist, the system returns a 409 error. If the email is new, the flow continues.

Next, the Auth Controller sends the password to the Crypto module, which it will be hashed by using bcrypt with a salt round of 10. The hashed password is then stored in the Database with the user's email and first name with an INSERT operation. After the account is created, the system enters the issue-token. The Auth Controller generating a random token using crypto.randomBytes, hashing it with SHA-256, and storing it in the Database through an RPC call with a expiry time one hour. Finally, the Email process sends a verification link to the user's email address. This diagram shows five participants working together across two distinct phases, with the Auth Controller acting as the central coordinator between all other components.

Sequence Diagram 2: Admin Page Request Processing

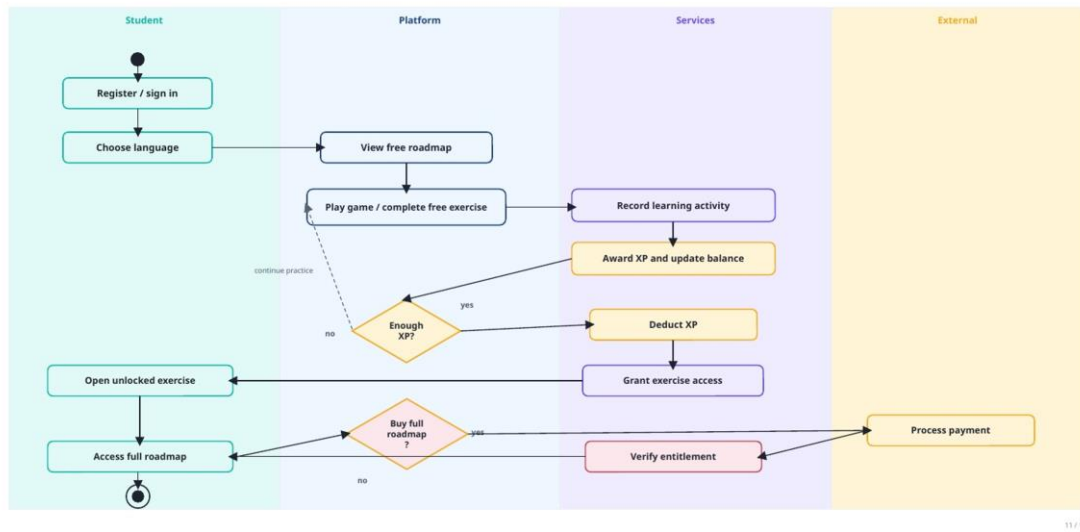
The second sequence diagram shows an admin action from the frontend all the way to the database and back. It starts when an admin user clicks an action on the Admin Page, such as updating an exercise status. The Admin Page sends an HTTP PATCH request to the Express Server using axios with a JWT cookie for authentication.

The Express Server then enters the pipeline phase, where the request passes through middleware including CORS, JSON parsing, cookie parsing, and route matching. Once the correct route is found, the server dispatches the request to the Admin Controller by calling the right function, in this case changeExerciseStatus. The Admin Controller communicates with Supabase in two ways, shown in the alt fragment: it either calls a stored RPC function like change_admin_exercise_status, or it performs a direct table operation using the Supabase client. Supabase returns a data and error object back to the Admin Controller. The controller wraps this into a consistent JSON response using handleControllerError for any exceptions, and sends it back to the Admin Page. The Admin Page then updates the UI by showing a toast notification, updating the state, and rerendering the component. This diagram clearly shows the five-layer request lifecycle from user click to database and back.

6.5 Activity Diagram

Student learning, XP and paid-content access

Swimlane activity flow showing the normal path, the XP unlock path and the payment path.



The activity diagram is divided into four swimlanes — Student, Platform, Services, and External — to map the full learning flow. The student registers, picks a language, and views the free roadmap. They then enter a loop of playing games and completing free exercises, with each activity earning XP. A decision point checks whether enough XP has been collected. If not, the student continues practising. If yes, XP is deducted and the exercise is unlocked. A second decision offers buying the full roadmap, which routes through an external payment service. The diagram captures three paths: free use, XP-based unlocking, and direct purchase.

7. Design Patterns Used

We applied several design patterns in our project to make the code more clean, flexible, scalable and easier to maintain. Each pattern was chosen for a specific reasons, not just to use patterns for the sake of it. Below we explain each pattern and why it fits our project.

Pattern	Where Used	Why We Used It
Singleton	DatabaseConnection class	We need only one single database connection shared across the entire application. The Singleton pattern makes sure only one instance is created and provides a global access point to it.
Factory Method	GameFactory class	The platform has different types of games (snake, ping_pong, tic_tac_toe and many more). The Factory pattern lets us create the right game object based on the game type without the rest of the code knowing the details of each game class.
Strategy	RecommendationEngine class	The recommendation algorithm can work in different ways: rule-based, frequency-based, or weakness-focused. The

		Strategy pattern lets us swap between these algorithms at runtime without changing the engine class itself.
Observer	XPManager and UI components	When a user earns XP, multiple parts of the UI need to update: the XP counter, the progress bar, and the unlock notifications. The Observer pattern lets the XPManager notify all these components automatically when the XP value changes.
Adapter	Reusable Library integration	Our reusable recommendation library has its own interface. The Adapter pattern wraps this library so it fits the interface that the main application expects, allowing the library to be used without modifying either side.

Using these very patterns follows the SOLID principles taught in this module before. The Strategy pattern provides the Open/Closed Principle because we can add new recommendation algorithms without modifying existing code we have. The Observer pattern supports the Single Responsibility Principle because each UI component handles its own update logic. The Factory pattern supports the Dependency Inversion Principle because the application depends on an game interface rather than concrete game classes (Gamma et al., 1994).

8. Reusable Code Library

8.1 Purpose and Functionality

Our reusable library is a Recommendation Engine Library. Its purpose is to analyze a user's learning history and generate suggestions for what to study next. The library is designed to work independently of the application, meaning it can be used in any learning platform.

8.2 Library Architecture

Component	Class	Responsibility
Data Model	ActivityRecord, UserProfile	Defines the structure of input data (activities, scores, topics)
Analyzer	StrengthAnalyzer, WeaknessAnalyzer	Processes activity data to find patterns in user performance
Strategy Interface	RecommendationStrategy	Abstract interface that all recommendation algorithms implement
Concrete Strategies	RuleBasedStrategy, FrequencyStrategy	Specific algorithms that generate recommendations differently
Engine	RecommendationEngine	Main entry point that takes a profile and a strategy, returns suggestions

8.3 How It Integrates with CodeQuest

In CodeQuest, the library is imported as a module. The application collects user activity data (exercise results, game scores) from the database, converts it into the library's ActivityRecord format using an Adapter, and at the end passes it to the RecommendationEngine. The engine returns a list of suggested topics, which the application will be displayed on the user's dashboard.

8.4 Reusability

The library is reusable because it is not dependent on CodeQuest's database, user interface, or any other part of the main application. It only depends on its own data models (ActivityRecord, UserProfile). Any application that provides a list of activity records can use this library to generate recommendations. The library is packaged as a separate modules in the repository under the /ReusableLibrary folder, with its own README, unit tests, and usage examples.

9. Demonstration

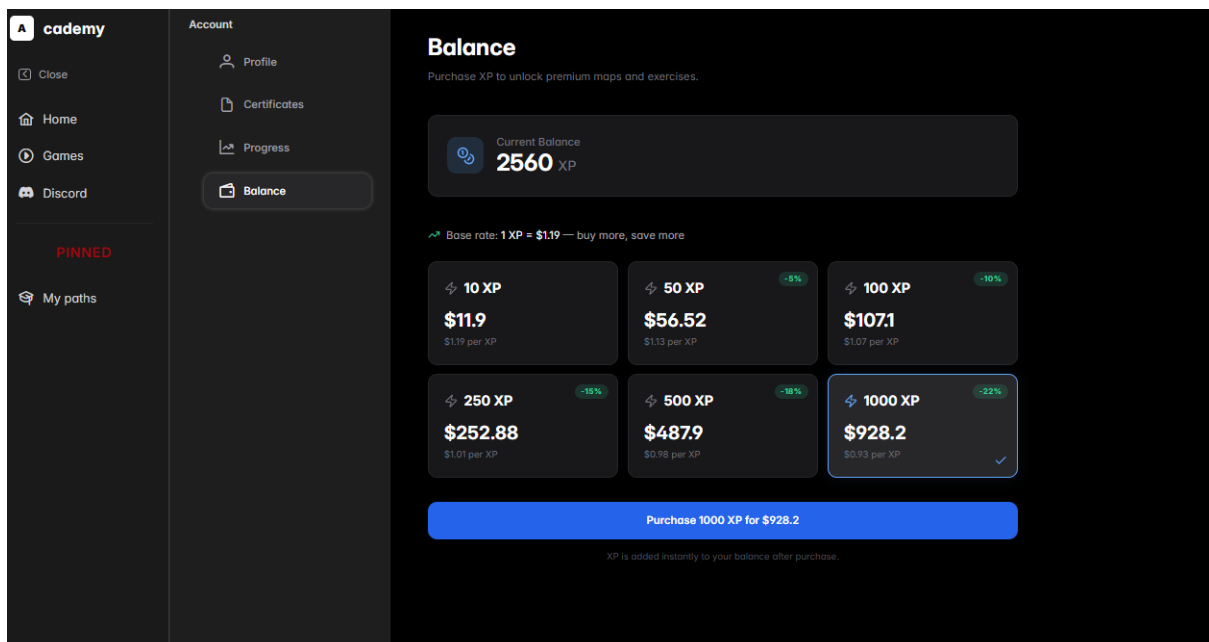
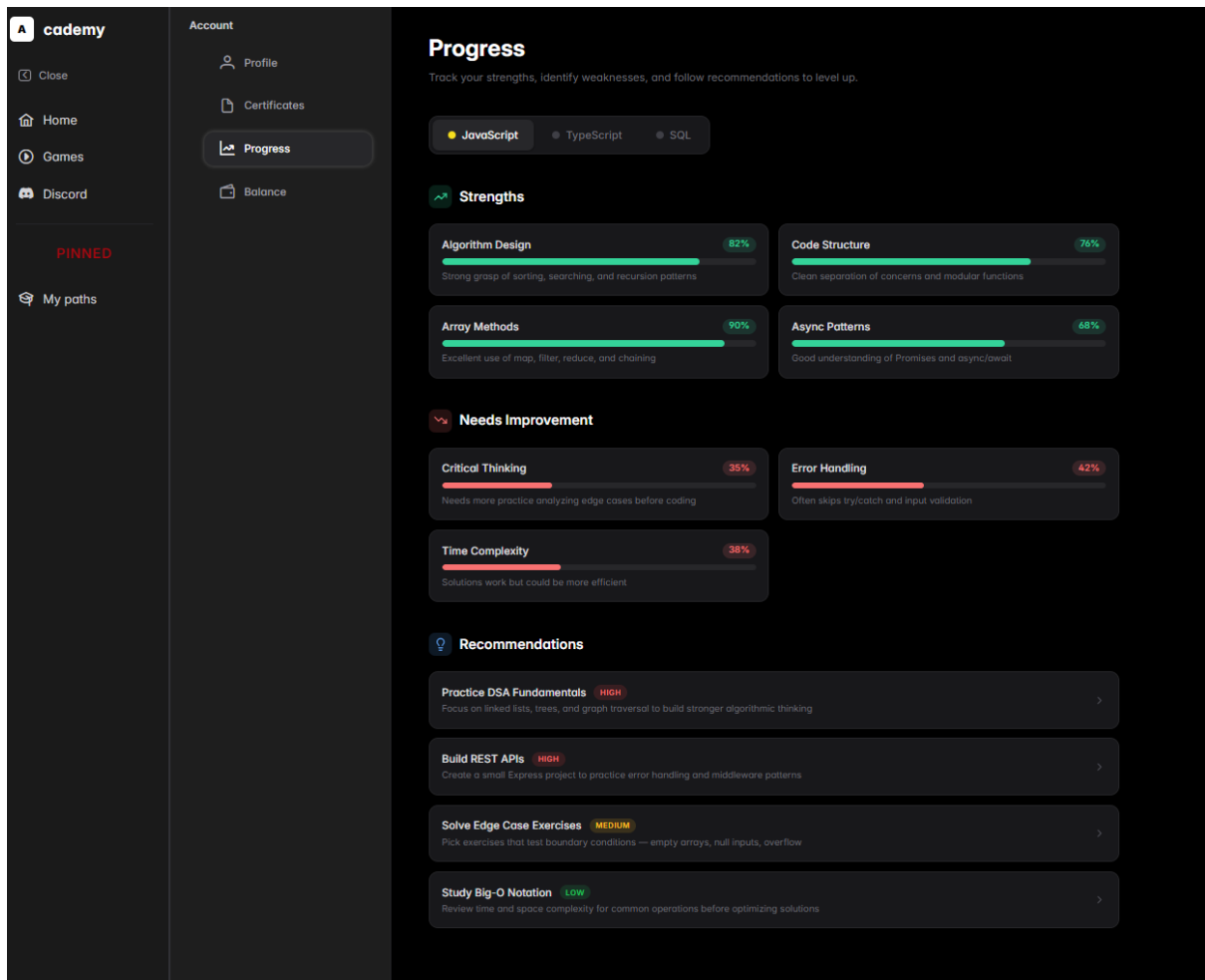
A video demonstration of the working application has been recorded and uploaded to [Google drive]. The video is [3–5] minutes long and covers the following:

- A brief introduction to the project idea and its purpose
- A walkthrough of the main screens: login, roadmap selection, exercise view, game screen, and dashboard
- A live demonstration of completing an exercise and seeing the progress update
- A live demonstration of playing a game, earning XP, and unlocking a paid exercise
- A brief look at the recommendation feature showing personalized suggestions

Video link:

https://drive.google.com/file/d/1e_124-ic18mkIXYGGLfrEqC83Ok00rMx/view?usp=drive_link

9.1 Screenshots



The screenshots above show the key screens of the application in its current state. The interface follows a clean and simple design so that users can navigate without needing a tutorial, which was one of our goals making as simple as possible.

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education.

Fowler, M. (2003) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston: Addison-Wesley.

Pressman, R.S. and Maxim, B.R. (2020) *Software Engineering: A Practitioner's Approach*. 9th edn. New York: McGraw-Hill.

Bass, L., Clements, P. and Kazman, R. (2021) *Software Architecture in Practice*. 4th edn. Boston: Addison-Wesley.

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. London: Prentice Hall.

Cohn, M. (2004) *User Stories Applied: For Agile Software Development*. Boston: Addison-Wesley.

Freeman, E., Robson, E., Bates, B. and Sierra, K. (2020) *Head First Design Patterns*. 2nd edn. Sebastopol: O'Reilly Media.

Larman, C. (2004) *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd edn. Upper Saddle River: Prentice Hall.